
Data Structures

BS (CS) _Fall_2025

Lab_09 Manual



Learning Objectives:

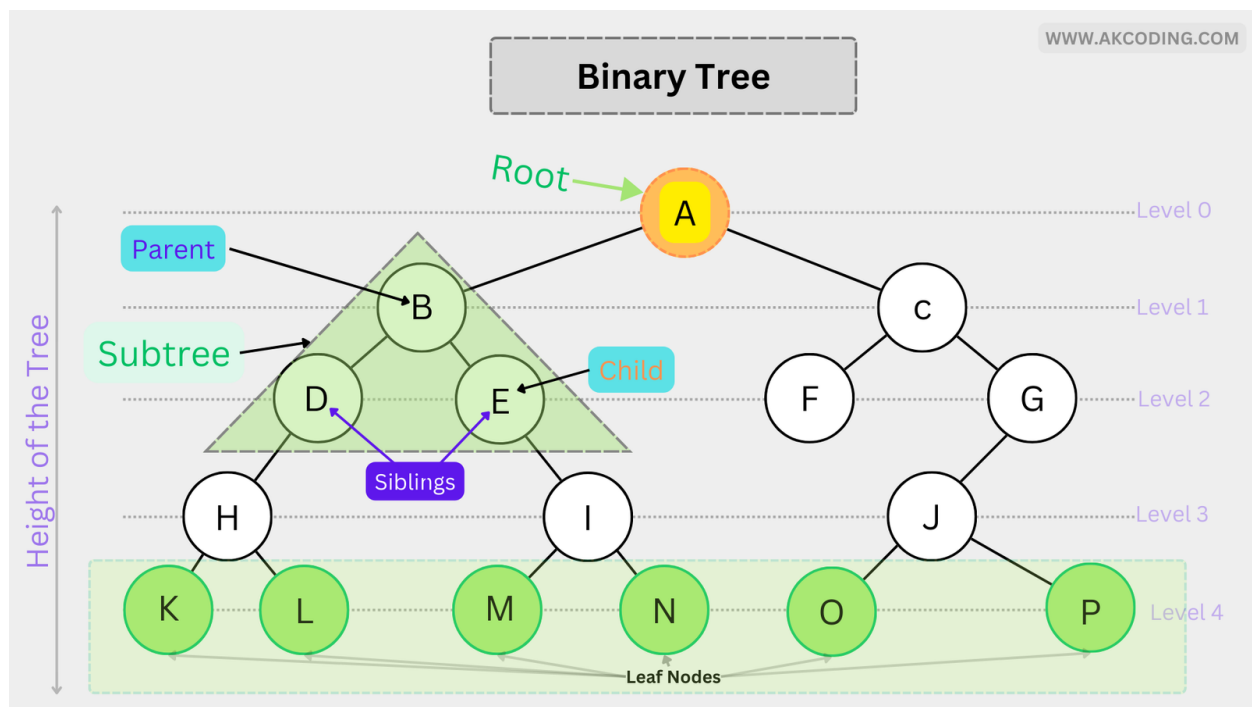
1. Binary Tree

Binary Tree

A Binary Tree is a hierarchical data structure in which each node can have at most two children: a left child and a right child.

Unlike Binary Search Trees (BSTs), a Binary Tree does not maintain any ordering property between the left and right subtrees.

Binary Trees are the foundation for advanced data structures like heaps, expression trees, and Huffman trees.



Node Structure

A Binary Tree can be implemented using a linked structure, where each node contains:

- data : stores the value or information
- left : pointer/reference to the left child
- right : pointer/reference to the right child

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

```
Node(int val) {  
    data = val;  
    left = right = nullptr;  
}  
};
```

Binary Tree Properties

- The topmost node is called the root.
- Each node may have 0, 1, or 2 children.
- Nodes with no children are called leaf nodes.
- The height of a tree is the number of edges on the longest path from the root to a leaf.

Types of Binary Trees

- **Full Binary Tree:** Every node has either 0 or 2 children.
- **Complete Binary Tree:** All levels are completely filled except possibly the last, which is filled from left to right.
- **Perfect Binary Tree:** All internal nodes have 2 children and all leaves are at the same level.
- **Skewed Binary Tree:** All nodes are aligned to one side (left or right).

Basic Binary Tree Operations

Insertion

Concept

Insertion in a Binary Tree is not based on order (like BST).

Instead, new nodes are added in the first available position — filling the tree from left to right.

Steps

- If the tree is empty → create a new node as the root.
- If the current node has no left child, insert there.
- Else if the current node has no right child, insert there.
- Otherwise, move to the left subtree and repeat recursively.

Code

```
Node* insert(Node* root, int val) {  
    if (root == nullptr)  
        return new Node(val);
```

```

    if (root->left == nullptr)
        root->left = new Node(val);
    else if (root->right == nullptr)
        root->right = new Node(val);
    else
        insert(root->left, val); // continue to next level

    return root;
}

```

Traversals

Traversal means visiting all nodes in a tree in a specific order.

There are three standard depth-first traversals.

A. Inorder Traversal (Left → Root → Right)

Steps

- Visit the left subtree.
- Visit (print) the current node.
- Visit the right subtree.

```

void inorder(Node* root) {
    if (root == nullptr) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

```

B. Preorder Traversal (Root → Left → Right)

Steps

- Visit (print) the current node.
- Traverse the left subtree.
- Traverse the right subtree.

```

void preorder(Node* root) {
    if (root == nullptr) return;
    cout << root->data << " ";
}

```

```
preorder(root->left);
preorder(root->right);
}
```

C. Postorder Traversal (Left → Right → Root)

Steps

- Visit the left subtree.
- Visit the right subtree.
- Visit (print) the current node.

```
void postorder(Node* root) {
    if (root == nullptr) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

3. Searching

Concept

In a Binary Tree, since there is no ordering rule, the search must check both subtrees recursively.

Steps

- If the tree is empty → return false.
- If current node's data matches → return true.
- Recursively search the left and right subtrees.
- If found in either → return true.

Code

```
bool search(Node* root, int key) {
    if (root == nullptr) return false;
    if (root->data == key) return true;

    // search both subtrees
    return search(root->left, key) || search(root->right, key);
}
```

4. Deletion

Concept

In a Binary Tree, deletion is different from a Binary Search Tree because there is no key ordering.

To maintain the complete shape of the tree (filled from left to right), we delete a node by replacing it with the deepest node (bottom-most, right-most node) and then removing that deepest node.

This ensures that the structure of the binary tree remains balanced and no gaps are created.

Steps

DeleteNode(root, key)

1. **Check for empty tree:**
 - If root is NULL, display “Tree is empty” and stop.
2. **Search for the target node:**
 - Recursively traverse the tree (both left and right subtrees) to find the node whose data matches the given key.
 - If not found, display “Value not found” and stop.
3. **Find the deepest node:**
 - Traverse to the bottom-most, right-most node of the tree.
 - This node is chosen because deleting it will not affect the completeness of the tree.
4. **Copy data:**
 - Replace the data of the target node with the data of the deepest node.
 - The target node now contains the value of the deepest node.
5. **Delete the deepest node:**
 - Traverse the tree again to locate the deepest node.
 - Once found, remove it by setting the corresponding pointer (left or right) to NULL.
6. **Display confirmation message:**
 - Print “Node deleted successfully” after the operation completes.

Time Complexities

Operation	Best Case	Average Case	Worst Case	Description
Insertion	$O(1)$	$O(\log n)$	$O(n)$	Insert node at first empty spot
Search	$O(1)$	$O(\log n)$	$O(n)$	Visit both subtrees recursively
Traversal	$O(n)$	$O(n)$	$O(n)$	Visit every node once

Operation	Best Case	Average Case	Worst Case	Description
Deletion	$O(1)$	$O(\log n)$	$O(n)$	Delete deepest node

Learning Outcomes

After completing this lab, students should be able to:

- Construct and manipulate Binary Trees using linked nodes
- Implement recursive insertion, traversal, and property functions
- Differentiate between Binary Tree and Binary Search Tree
- Analyze how recursion simplifies tree operations
- Understand how tree shape affects performance